ESTRUCTURA FINAL DEL PROYECTO

```
Carpeta contenedora del proyecto/
│
├── rclone/
│   └── rclone.exe
│
├── uploads/
│
├── index.html
├── style.css
├── app.js
├── renderer.js 25
├── main.js
├── server.js
├── package.json 25
```

index.html

```
<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Subida a Google Drive</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<div class="card">
  <header>
    <h1>Sube tu archivo a Drive</h1>
  </header>

  <div id="upload-box">
    <strong>Arrastra tu archivo aquí</strong>
    <p>o haz clic para seleccionar</p>
    <input type="file" id="file-input">
  </div>

  <div id="info">
    <div class="file-info">
      <p><strong>Archivo:</strong> <span id="file-name"></span></p>
      <p><strong>Tamaño:</strong> <span id="file-size"></span></p>
    </div>

    <div class="actions">
      <button class="secondary" id="change-btn">Cambiar archivo</button>
      <button id="send-btn">Subir archivo</button>
    </div>

    <input id="link" readonly placeholder="El link de Drive aparecerá aquí">
  </div>

  <div id="result"></div>
</div>

<script src="app.js"></script>
</body>
</html>
```

style.css

```
@import
url('https://fonts.googleapis.com/css2?family=Poppins:wght@300;400;600&display=swap');

:root {
  --bg:#000;
  --card:rgba(50,50,50,.9);
  --text:#fff;
  --muted:#aaa;
  --border:rgba(255,255,255,.2);
  --radius:16px;
}

* {
  box-sizing:border-box;
  font-family:'Poppins',sans-serif;
}

body {
  margin:0;
  min-height:100vh;
  background:var(--bg);
  display:flex;
  justify-content:center;
  align-items:center;
  color:var(--text);
}

.card {
  width:420px;
  padding:28px;
  background:var(--card);
  border-radius:var(--radius);
  border:1px solid var(--border);
  backdrop-filter:blur(14px);
  text-align:center;
}

#upload-box {
  border:2px dashed var(--border);
  padding:36px 20px;
  border-radius:var(--radius);
  cursor:pointer;
}

input[type="file"] {
  display:none;
```

```
}

.actions {
  display:flex;
  gap:12px;
  margin-top:20px;
}

button {
  flex:1;
  padding:12px;
  border:none;
  border-radius:10px;
  cursor:pointer;
  font-weight:600;
  color:#fff;
  background:#00bfff;
}

button.secondary {
  background:#222;
  color:#00bfff;
}

#link {
  margin-top:14px;
  width:100%;
  padding:10px;
  border-radius:8px;
  border:1px solid var(--border);
  background:rgba(0,0,0,.3);
  color:var(--text);
  font-size:.75rem;
}
```

renderer.js

```
const { contextBridge, ipcRenderer } = require('electron')

contextBridge.exposeInMainWorld('api', {
  uploadFile: () => ipcRenderer.invoke('upload-file')
})
```

app.js

```
const box = document.getElementById('upload-box')
const fileInput = document.getElementById('file-input')
const infoDiv = document.getElementById('info')
const nameSpan = document.getElementById('file-name')
const sizeSpan = document.getElementById('file-size')
const changeBtn = document.getElementById('change-btn')
const sendBtn = document.getElementById('send-btn')
const linkInput = document.getElementById('link')
const resultDiv = document.getElementById('result')

let selectedFile = null
infoDiv.style.display = 'none'

const formatSize = bytes => {
  const k = 1024
  const sizes = ['Bytes','KB','MB','GB']
  const i = Math.floor(Math.log(bytes)/Math.log(k))
  return (bytes/Math.pow(k,i)).toFixed(2)+' '+sizes[i]
}

const handleFile = file => {
  selectedFile = file
  nameSpan.textContent = file.name
  sizeSpan.textContent = formatSize(file.size)
  box.style.display = 'none'
  infoDiv.style.display = 'block'
  resultDiv.textContent = ''
  linkInput.value = ''
}

box.onclick = () => fileInput.click()

fileInput.onchange = e => {
  if(e.target.files.length) handleFile(e.target.files[0])
}

changeBtn.onclick = () => {
  selectedFile = null
  fileInput.value = ''
  infoDiv.style.display = 'none'
  box.style.display = 'block'
}

sendBtn.onclick = async () => {
  if(!selectedFile) return alert('Selecciona un archivo')
```

```
  const CHUNK = 1024 * 1024 * 5
  const uploadId = crypto.randomUUID()
  let offset = 0

  sendBtn.disabled = true
  changeBtn.disabled = true
  resultDiv.textContent = 'Subiendo...'

  try {
    while (offset < selectedFile.size) {
      const chunk = selectedFile.slice(offset, offset + CHUNK)
      const isLast = offset + CHUNK >= selectedFile.size

      const res = await fetch('/upload', {
        method: 'POST',
        headers: {
          'x-api-key': 'drive-uploader',
          'x-upload-id': uploadId,
          'x-file-name': selectedFile.name,
          'x-last-chunk': isLast
        },
        body: chunk
      })

      const data = await res.json()
      if (!data.success) throw new Error(data.message)

      offset += CHUNK
      resultDiv.textContent =
        `Subiendo ${Math.floor(offset / selectedFile.size * 100)}%`

      if (isLast) {
        resultDiv.textContent = '¡Subida exitosa!'
        linkInput.value = data.driveLink
        sendBtn.disabled = false
        changeBtn.disabled = false
      }
    }
  } catch (e) {
    resultDiv.textContent = 'Error: ' + e.message
    sendBtn.disabled = false
    changeBtn.disabled = false
  }
}
```

main.js

```
const { app, BrowserWindow, ipcMain, dialog } = require('electron')
const { exec } = require('child_process')
const path = require('path')

function createWindow () {
  const win = new BrowserWindow({
    width: 520,
    height: 420,
    title: 'Drive Uploader',
    icon: path.join(__dirname, 'assets', 'icon.ico'), // opcional
    webPreferences: {
      preload: path.join(__dirname, 'renderer.js'),
      contextIsolation: true
    }
  })

  win.loadFile('index.html')
}

ipcMain.handle('upload-file', async () => {
  const { canceled, filePaths } = await dialog.showOpenDialog({
    properties: ['openFile']
  })

  if (canceled) return null

  const filePath = filePaths[0]
  const fileName = path.basename(filePath)

  //  rclone relativo a la carpeta contenedora del proyecto
  const rclonePath = `"${path.join(__dirname, 'rclone', 'rclone.exe')}"`
  const remote = 'gdrive:/drive-uploads/'

  return new Promise((resolve, reject) => {

    //  Subir archivo
    exec(
      `${rclonePath} copy "${filePath}" ${remote}`,
      { shell: true },
      err => {
        if (err) {
          reject(err)
          return
        }

        // Obtener link
```

```
        exec(
          `${rclonePath} link ${remote}${fileName}`,
          { shell: true },
          (err2, stdout) => {
            if (err2) {
              reject(err2)
              return
            }

            let link = stdout.trim()

            // Convertir a descarga directa (opcional)
            if (link.includes('open?id=')) {
              const id = link.split('open?id=')[1]
              link = `https://drive.google.com/uc?export=download&id=${id}`
            }

            resolve(link)
          }
        )
      }
    )
  })
})

app.whenReady().then(createWindow)
```

server.js

```
const express = require('express')
const { execFile } = require('child_process')
const path = require('path')
const fs = require('fs')
const crypto = require('crypto')

const app = express()
const PORT = 3000

const UPLOAD_DIR = path.join(__dirname, 'uploads')
const RCLONE_PATH = path.join(__dirname, 'rclone', 'rclone.exe')
const REMOTE_FOLDER = 'drive-uploads'

if (!fs.existsSync(UPLOAD_DIR)) fs.mkdirSync(UPLOAD_DIR)

const receiveChunk = (req, file) => new Promise((res, rej) => {
  const stream = fs.createWriteStream(file, { flags: 'a' })
  req.pipe(stream)
  req.on('end', res)
  req.on('error', rej)
})

const gatekeeper = (req, res, next) => {
  if (req.headers['x-api-key'] === 'drive-uploader') return next()
  res.status(401).json({ success: false })
}

app.use(express.static(__dirname))

app.post('/upload', gatekeeper, async (req, res) => {
  try {
    const id = req.headers['x-upload-id'] || crypto.randomUUID()
    const name = req.headers['x-file-name']
    const last = req.headers['x-last-chunk'] === 'true'
    if (!name) return res.json({ success: false })

    const dir = path.join(UPLOAD_DIR, id)
    if (!fs.existsSync(dir)) fs.mkdirSync(dir)
```

```
    const file = path.join(dir, name)
    await receiveChunk(req, file)

    if (!last) return res.json({ success: true })

    execFile(RCLONE_PATH, ['copy', file, `gdrive:/${REMOTE_FOLDER}/`], () => {
      execFile(
        RCLONE_PATH,
        ['link', `gdrive:/${REMOTE_FOLDER}/${name}`],
        (err, out) => {
          if (err) return res.json({ success: false })
          res.json({ success: true, driveLink: out.trim() })
        }
      )
    })
  } catch (e) {
    res.json({ success: false, message: e.message })
  }
})

app.listen(PORT, () => {
  console.log(`Servidor activo en http://localhost:${PORT}`)
})
```

package.json

```
{
  "name": "drive-uploader",
  "version": "1.0.0",
  "main": "main.js",
  "scripts": {
    "start": "npx electron ."
  },
  "dependencies": {
    "electron": "^30.0.0",
    "express": "^5.2.1"
  }
}
```